Simulare N-Body: paralelizare și optimizare

Model: interacțiuni gravitaționale între N particule (forță ~ $1/r^2$)

Aplicații: astrofizică (dinamică stelară), simulări cosmologice, plasma / molecular dynamics

Obiectiv: accelerează pasul dominant (computeForces) folosind Pthreads / MPI / CUDA

De ce paralelizare?

Complexitate și cost

- Suma directă a forțelor: $O(N^2)$ per pas
- Pentru N=3000: ~9 milioane interacțiuni / pas
- Pentru mii de pași → zeci/sute de secunde pe CPU

Limite teoretice

Chiar dacă paralelizăm 100% din computeForces, speedup-ul e limitat de:

- partea secvențială (inițializare, I/O)
- overhead de sincronizare / comunicare
- dezechilibru de încărcare

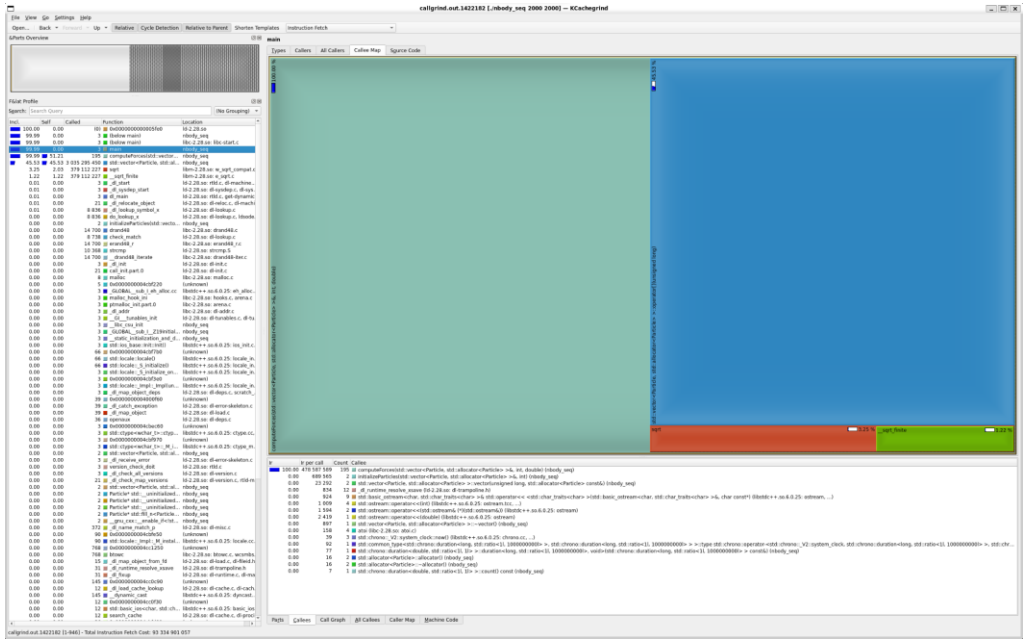
Speedup $S(p) = T(1) / T(p)$ | Eficiență $E(p) = S(p) / p$

$S(p) \leq 1 / ((1 - f) + f/p)$ ⇒ dacă $f < 1$, speedup-ul se plafonează.

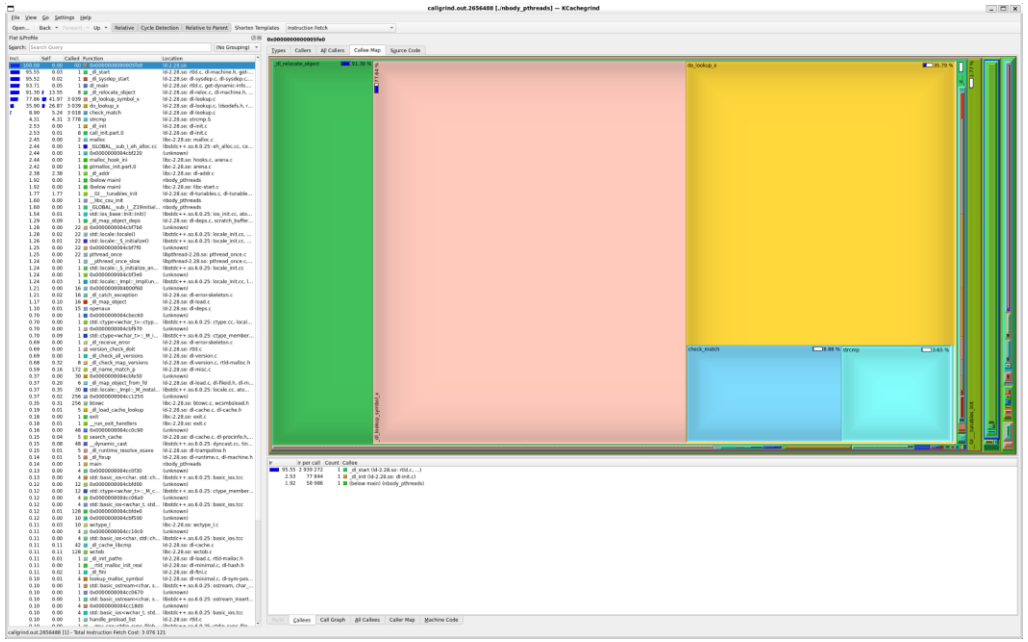
Scop practic: reducerea timpului total prin creșterea paralelismului și reducerea overhead-ului (thread creation, sync, comunicație).

Ce paralelizăm? (profilare + identificarea „hotspot”-urilor)

Profilare cu KCachegrind: aproape tot costul e în bucla dublă i–j (computeForces).



Secvențial (profil)



Pthreads (profil)

```
for (int i = 0; i < n; i++) {
  double fx=0, fy=0, fz=0;
  for (int j = 0; j < n; j++) {
    if (i==j) continue;
    // dx,dy,dz; dist; invDist3
    // F = G * mi * mj * invDist3
    fx += F * dx; fy += F * dy; fz += F * dz;
  }
}
```

Hotspot: bucla dublă i–j (forțele între toate perechile) — independentă pe i

De ce se poate paraleliza?

- Pentru un pas: fiecare i calculează suma forțelor independent (read-only pe particulele j)
- Update-ul vitezei/poziției e tot independent pe i ⇒ paralelizare pe particule (range decomposition).

Abordare: MPI (distributed-memory)

Idee (decompunere pe particule)

- Fiecare proces (rank) primește un interval [start..end)
- Calculează forțele pentru particulele lui folosind toate particulele globale
- După update, procesele schimbă noile particule → MPI_Allgather / Allgatherv
- Scalare pe noduri diferite (rețea) – dar cost de comunicație

```
MPI_Bcast(particles, N*sizeof(Particle), MPI_BYTE, 0, COMM);
computeRange(rank, size, N, start, end);

for (step=0; step<STEPS; step++) {
  for (i=start; i<end; i++) {
    // bucla j=0..N-1 (read-only)
    // update v si x pentru particula i
  }
  // sincronizare / schimb de date
  MPI_Allgather(...); // sau MPI_Allgatherv cand bucatile difera
}
```

Flux de date (per pas)

Rank 0: i in [start,end)

Rank 1: i in [start,end)

Rank 2: i in [start,end)

MPI_Allgather / Allgatherv: fiecare rank primește particulele actualizate

Notă: MPI_Allgather cere aceeași dimensiune trimisă de fiecare rank; dacă N nu e divizibil cu size, se folosește MPI_Allgatherv (recvcounts[]).

Abordare: Pthreads (threading manual)

Idee

- Împărțim intervalul de particule [0..N) în THREADS bucăți
- Fiecare thread calculează forțele pentru [start..end)
- Apoi o a doua fază: update poziții pe același interval
- Sincronizare: pthread_join (barieră) după fiecare fază

```
// per pas: 2 faze
pthread_create(..., computeForcesThread, &args[t]);
... pthread_join(...);

pthread_create(..., updatePositionsThread, &args[t]);
... pthread_join(...);

// fiecare thread procesează [start, end)
```

Din nbody_pthreads.cpp (creare/join la fiecare pas)

Trade-off-uri și îmbunătățiri

- Control fin (afinitate, bariere custom), dar cod mai lung și predispus la bug-uri.
- Overhead mare aici: thread-urile sunt create/distruse la fiecare pas (de 2×/pas).
- Acces concurent: fiecare thread scrie în p[i] (disjunct) → OK; citire pe p[j] (shared) → OK.

Abordare: CUDA (GPU Parallelization)

Idee

•Problema **N-Body** este **data-parallel**: fiecare particulă poate fi actualizată independent într-un pas de timp.
•CUDA permite lansarea a mii de thread-uri GPU pentru a calcula în paralel:

- **forțele gravitaționale**
- **actualizarea vitezelor și pozițiilor**

```
// 1 thread GPU -> 1 particula i
__global__ void nbody_step(Particle* p, int N, double dt) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i >= N) return;

  double fx=0, fy=0, fz=0;
  for (int j = 0; j < N; j++) if (i != j) {
    double dx=p[j].x-p[i].x, dy=p[j].y-p[i].y, dz=p[j].z-p[i].z;
    double inv = rsqrt(dx*dx + dy*dy + dz*dz + 1e-9);
    double F = G * p[j].mass * inv*inv*inv;
    fx += F*dx; fy += F*dy; fz += F*dz;
  }

  p[i].vx += dt * fx;  p[i].vy += dt * fy;  p[i].vz += dt * fz;
  p[i].x  += dt * p[i].vx; p[i].y += dt * p[i].vy; p[i].z += dt *
p[i].vz;
}
```

Ce se paralelizează

•Bucla dominantă ca timp este:

- calculul accelerației pentru fiecare corp (forțe între corpuri)

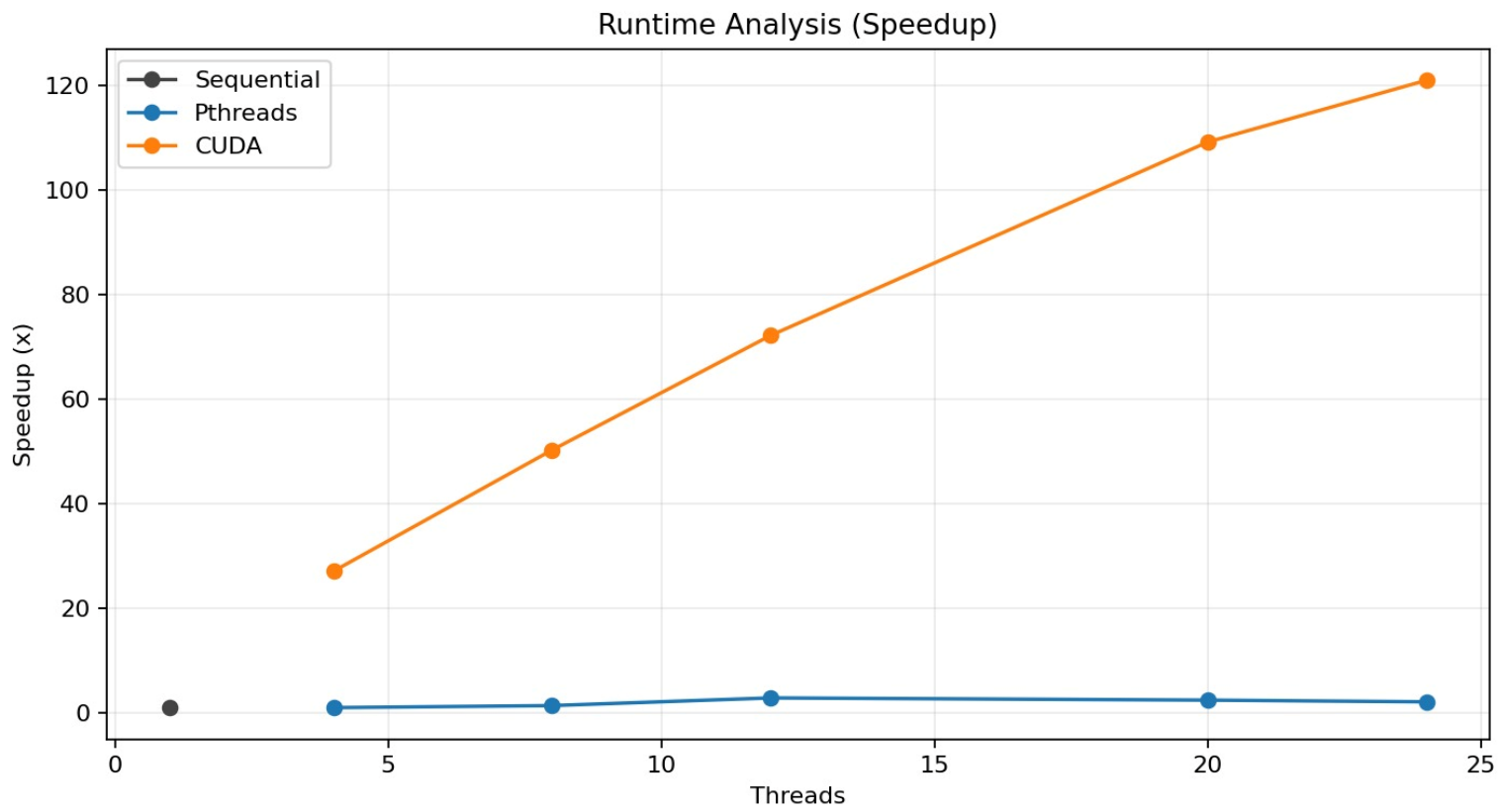
•Pe GPU:

- **un thread = o particulă i**
- thread-ul calculează contribuțiile tuturor particulelor **j** asupra lui **i**
- complexitate per pas: **$O(N^2)$** (dar foarte bine distribuită)

Paralelizare: fiecare thread calculează forțele și actualizează poziția pentru o singură particulă *(i)*, iterând peste toate *(j)*.

Rezultate: Speedup

Benchmark local: N=3000, STEPS=3000 (aceeași configurație pentru toate rulările)



Mediu de referință:

•Implementare secvențială: **223.815 s**

Observații Pthreads:

•Speedup maxim ≈ **2.9×** la **12 thread-uri**

•După 12 thread-uri apare **degradare de performanță**

•Cauze: sincronizare, contention pe memorie, overhead de thread management

Observații CUDA:

•Speedup foarte ridicat: până la **~121×**

•Scalare aproape monotonă odată cu creșterea paralelismului

•GPU exploatează masiv paralelismul de date specific problemei N-Body

Concluzie:

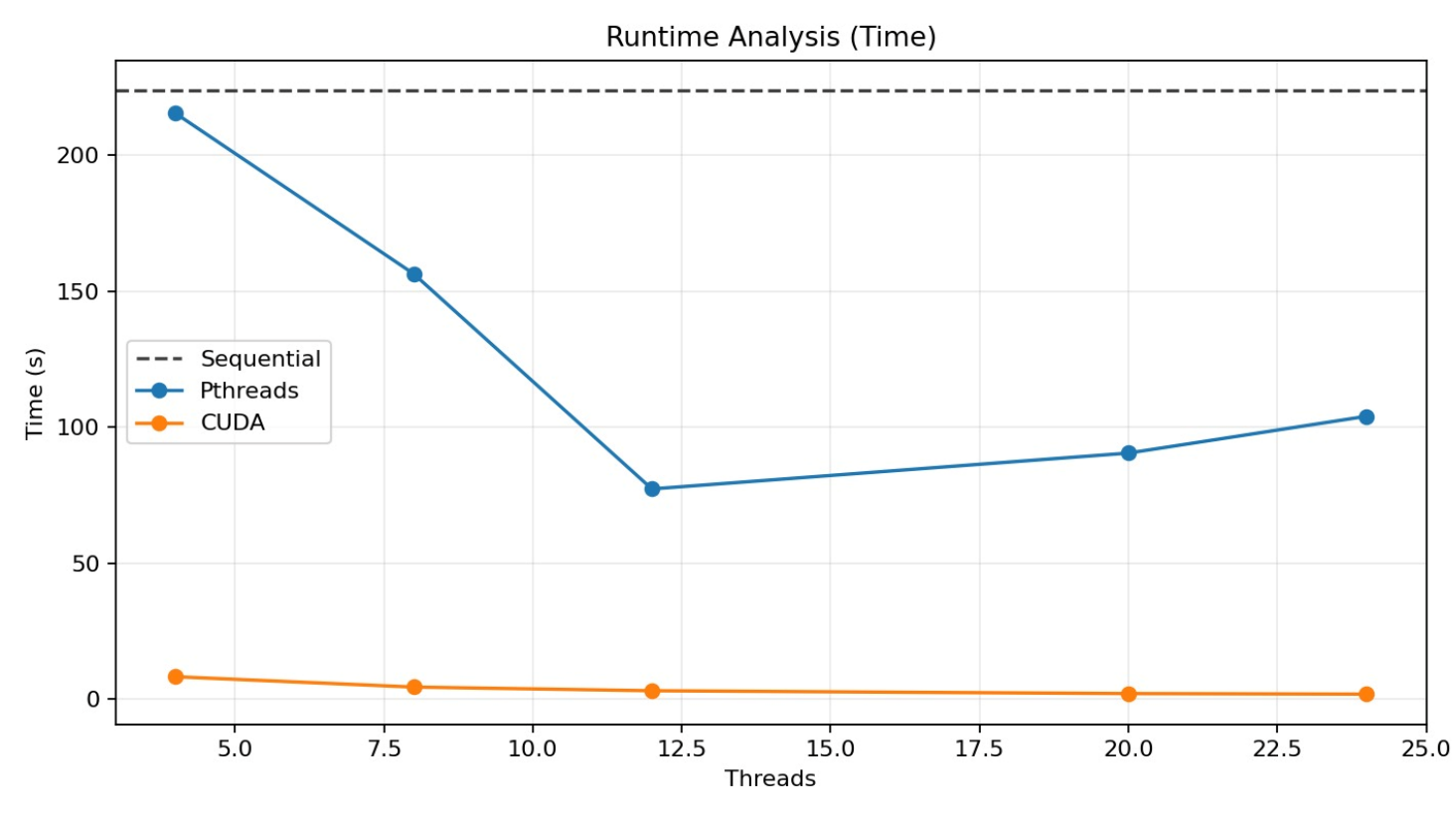
•N-Body este **puternic data-parallel**

•CPU multi-threaded are beneficii limitate

•GPU oferă cea mai eficientă accelerare

Rezultate: Eficiență

Eficiență E(p)=S(p)/p — cât de bine folosim fiecare thread/proces



Pthreads:

- Eficiență maximă ≈ **0.26**
- Scade rapid odată cu creșterea numărului de thread-uri
- Limitări impuse de:
  - acces concurent la date
  - latență memorie
  - overhead de sincronizare

CUDA:

- Eficiență > 1
- Explicată prin:
  - mii de thread-uri hardware
  - execuție masiv paralelă
  - model diferit față de CPU (SIMT)
- GPU nu se compară direct cu CPU-threads → rezultat așteptat

Ce am învățat

Concluzii principale

•**Varianta secvențială (Basic)** este punctul de referință, cu timp de execuție **223.815 s**, dar nu poate scala pentru dimensiuni mari ale problemei N-Body.

•**Pthreads** aduce un câștig real de performanță, însă **scalarea este limitată**:

- speedup maxim obținut: **~2.9×** la **12 thread-uri**
- după această valoare, performanța scade (ex: 20 și 24 thread-uri) din cauza **overhead-ului de sincronizare** și a **bottleneck-urilor de memorie**

•**CUDA** oferă cea mai bună performanță, datorită paralelismului masiv al GPU:

- speedup maxim: **~121×** (24 threads / configurație testată)
- timpul minim obținut: **~1.85 s**, ceea ce confirmă că problema este **highly data-parallel**